
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

In this work, we propose FLOWTIME, a general generative framework for probabilistic forecasting of multivariate timeseries data arising across scientific, industrial, and societal domains. Given historical measurements and, where available, future covariates, we cast forecasting as sampling from a learned conditional distribution over future trajectories, a formulation that applies uniformly whether observations are low- or high-dimensional and whether covariates are present or absent. Concretely, we decompose the joint distribution of future observations into a sequence of conditional densities, each modeled through a single shared flow that transports a simple base distribution to the next-observation distribution conditioned on the available context. We realize these transformations within the flow matching (FM) framework, which affords scalable and simulation-free learning that extends readily to the high-dimensional regimes typical of real-world data. By pairing this factorization with the FM objective, FLOWTIME inherits the broad strengths of autoregressive models—including strong extrapolation across varying horizons, compact model size, and well-calibrated uncertainty estimates—while retaining the capacity to represent the complex, multi-modal conditional distributions characteristic of modern transport-based generative models. We demonstrate the effectiveness and generality of FLOWTIME across a diverse suite of stochastic dynamical systems and real-world forecasting tasks spanning multiple domains, dimensionalities, and sampling frequencies.

1 INTRODUCTION

Probabilistic timeseries forecasting is a foundational problem in modern machine learning, in which the goal is to extrapolate how the state of a system evolves into the future given observational data, together with a faithful characterization of the associated uncertainty. Because virtually every measured process unfolds over time, this problem sits at the heart of an exceptionally broad range of scientific, industrial, and societal disciplines (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024).

A prominent and rapidly growing line of work approaches forecasting with deep generative models (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021). Under this view, prediction is framed generically as sampling from a future probability density conditioned on the past, a perspective that accommodates arbitrary predictive distributions rather than committing to a single point estimate. Diffusion models and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020), in particular, have recently advanced the state of the art across a variety of forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kollovieh et al., 2023; Meijer & Chen, 2024). Despite these strong results, diffusion models generally incur substantial computational cost during both training and inference, which can limit their applicability as data scale and dimensionality grow.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) is an emerging and increasingly general paradigm for generative modeling that generalizes and subsumes diffusion models while affording considerably more flexible design choices. In contrast to diffusion models, which rely on iterative stochastic denoising steps over a discretized trajectory, FM learns deterministic probability paths that transport an arbitrary base distribution directly onto the target distribution through continuous normalizing flows. Because these probability paths can be designed freely, FM

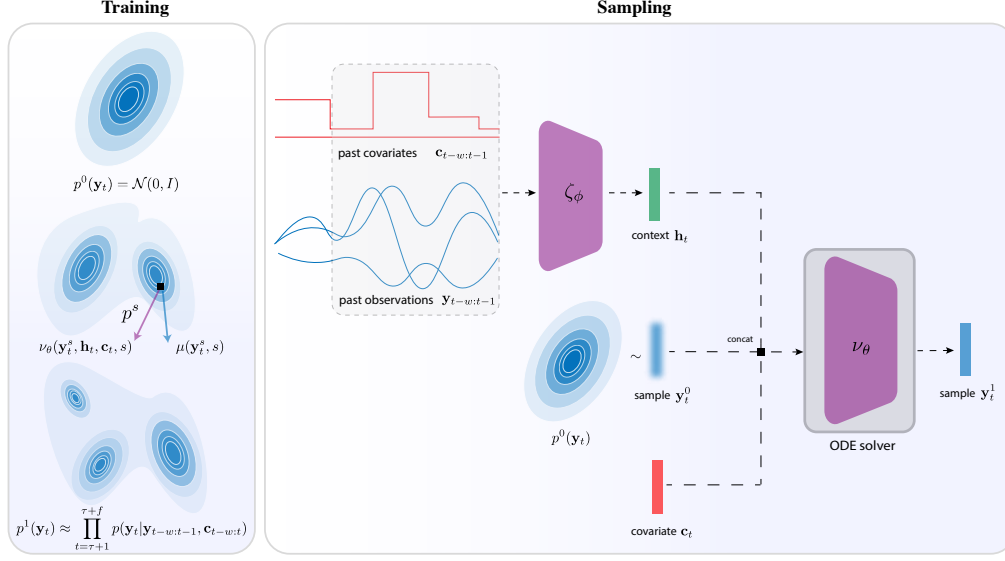


Figure 1: An overview of the training and sampling process underlying our general approach. During training, a probability path $(p^s)_{0 \leq s \leq 1}$ is constructed between a base distribution p^0 and the target distribution p^1 , and is generated by a velocity field μ . Training amounts to regressing μ with a neural network ν_θ that takes as input a sample from the probability path y_t^s at flow step s , a context vector $\mathbf{h}_t$ summarizing past observations and any available covariates, and the current covariate $\mathbf{c}_t$. The same components apply regardless of the observation dimensionality or forecast horizon. After training, sampling from the target distribution proceeds by first drawing from the base distribution p^0 and integrating the trained velocity field with an ODE solver until $s = 1$.

dispenses with the handcrafted noise schedules and lengthy ancestral sampling chains inherent to diffusion, yielding more efficient training and sampling across a wide range of data modalities.

FM has recently been brought to bear on timeseries modeling and probabilistic forecasting (Tamir et al., 2024; Hu et al., 2024; Kollovieh et al., 2024), where it delivers strong empirical results and markedly improved computational efficiency relative to diffusion models. Existing approaches, however, typically learn the conditional distribution of an entire fixed-horizon future window given a fixed-horizon context window. While convenient for fast training and sampling, this design tends to extrapolate poorly beyond the horizons seen during training and to produce miscalibrated uncertainty estimates. It also poses a harder optimization problem, since the model must learn intricate time-dependent probability paths along the flow dimension. This difficulty is highlighted by (Kollovieh et al., 2024), who introduce informed Gaussian-process priors to simplify and accelerate optimization; that remedy, however, is largely confined to the univariate setting and does not extend naturally to the broader multivariate regimes forecasting practitioners routinely encounter.

In this work, we introduce FLOWTIME, a simple and broadly applicable alternative that uses FM to learn probabilistic forecasting models scaling gracefully from univariate series to high-dimensional multivariate datasets, all without reliance on informed priors. Rather than modeling the entire future window at once as existing FM-based methods do, FLOWTIME decomposes forecasting into a sequence of conditional distributions, modeling the distribution of each future time step given past observations and any available covariates. A single shared flow transports a simple base distribution to the next-observation distribution, so the same mechanism applies uniformly across time steps, horizons, and dimensionalities. This autoregressive structure lets FLOWTIME accommodate arbitrary and variable forecasting horizons, extrapolate well beyond the conditions seen in training, and yield well-calibrated uncertainty estimates, all while simplifying the underlying optimization problem.

We show that FLOWTIME attains strong performance across both classical stochastic dynamical systems and a diverse set of real-world forecasting tasks, providing a practical, general-purpose solution to probabilistic timeseries forecasting.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) is a general paradigm for generative modeling that enables training of continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018) at an unprecedented scale and across arbitrary data domains. CNFs model data transformations as solutions to ordinary differential equations (ODEs), providing invertible mappings with tractable likelihoods.

As with CNFs, FM seeks a time-dependent diffeomorphic map on a data space Ω , called a flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$; here Ω is left generic and may denote data of any dimensionality. This flow transforms a sample $\mathcal{X}^0 \sim p$ from a source distribution p into a target sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ such that $\mathcal{X}^1 \sim q$ for some target distribution q . The flow is constructed by solving the following initial value problem:

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

where $\mu: [0, 1] \times \Omega \rightarrow \Omega$ is a vector field defining the velocity of the flow and generating a probability path $(p^s)_{0 \leq s \leq 1}$ where each p^s is a distribution over Ω with $p^0 = p$. The objective is thus to learn a valid vector field μ such that $p^1 = q$.

Unlike CNFs where the vector field is learned via likelihood maximization, requiring solving and differentiating through Eq. (1) during training, FM learns a vector field ν_θ with parameters θ by regressing over vector fields of fixed conditional probability paths. This enables defining a tractable objective function called conditional flow matching (CFM) defined as:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0,1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

Minimizing this objective enables sampling from q by solving Eq. (1) with the trained ν_θ . Because it avoids backpropagation through the ODE solver, FM supports stable and scalable training even for very high-dimensional generative tasks, as demonstrated in domains as varied as image and video modeling (Esser et al., 2024; Polyak et al., 2024). A rich array of probability-path designs and conditioning strategies has since been proposed, unifying a broad family of transport-based generative models under a single framework (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023).

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ denote an n -dimensional observation at time τ , where n is arbitrary so that the same formulation covers univariate and high-dimensional multivariate series alike. Given its l past measurements including the current observation, denoted $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, our objective is to produce probabilistic forecasts of the next f future values $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$, optionally conditioned on observed covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$ with $\mathbf{c}_\tau \in \mathbb{R}^c$; the covariate channel is entirely optional and may be omitted when no side information is available. Formally, we aim to learn to sample from the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ from a dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$ containing m instances.

We factorize this conditional distribution autoregressively across future time steps under a general Markov assumption of order w , giving:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where the history size $w \leq l$ is a free hyperparameter. Our objective thus reduces to learning the conditional distribution in Eq. (3). Rather than the maximum likelihood estimation customary for autoregressive models, we learn this conditional distribution within the flow matching framework (Lipman et al., 2022), which places no parametric restriction on the shape of the predictive density.

This autoregressive factorization confers several general advantages. First, it reduces the learning problem to modeling much simpler conditional distributions $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ instead of a complex high-dimensional joint distribution, a reduction that becomes increasingly valuable as the

dimensionality and horizon grow. Second, each per-step conditional can be trained within a teacher-forcing framework, enabling fully parallel training across time steps. Third, the Markov order w exposes a tunable trade-off between model capacity and computational cost that can be adapted to the demands of a given application. Finally, the formulation supports interpretable uncertainty quantification, since it explicitly models how predictive uncertainty propagates through time along the chain of conditional distributions.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For every future time step $t \in [\tau + 1, \tau + f]$, we define a target probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$ with marginals $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$, where $p_{\mathcal{D}}$ is the empirical distribution. This path is generated by a flow field $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ along s , defined for observations of arbitrary dimension n by the ordinary differential equation (ODE):

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a vector field generating the flow, chosen so that the resulting flow satisfies the continuity equation.

To obtain a tractable objective, we model the marginal probability path as a mixture of conditional probability paths:

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t | \mathbf{z}) p(\mathbf{z}) d\mathbf{z} \quad (5)$$

where $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is a conditioning random variable drawn from an arbitrary data coupling $\pi_{0,1}$, leaving substantial freedom in how the path is constructed. Theorem 2 in (Lipman et al., 2022) establishes that the conditional velocity fields $\mu(\mathbf{y}_t, s | \mathbf{z})$ associated with the conditional probability paths $p^s(\mathbf{y}_t | \mathbf{z})$ coincide with their marginal velocity field $\mu(\mathbf{y}_t, s)$.

Following (Tong et al., 2023), we adopt the particularly simple and general choice $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are samples from p^0 and p^1 , respectively. The conditional probability path is then:

$$p^s(\mathbf{y}_t | \mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

whose marginals satisfy $p^0(\mathbf{y}_t | \mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t | \mathbf{z}) = p^1(\mathbf{y}_t)$ as $\sigma^2 \rightarrow 0$. The corresponding conditional velocity field is then simply $\mu(\mathbf{y}_t, s | \mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This choice yields straight probability paths between source and target distributions in Euclidean space together with closed-form expressions, making training and sampling efficient at any scale. Learning this velocity field lets us sample from the target distribution $p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ by integrating the ODE in Eq. (4) until $s = 1$.

3.2 TRAINING

As in the standard FM setting, we learn this velocity field by regressing it onto a neural network ν_θ with parameters θ using the conditional flow matching objective in Eq. 2. To do so efficiently and in a manner agnostic to the choice of context encoder, we introduce a context vector $\mathbf{h}_t \in \mathbb{R}^h$ that summarizes the context window at time t :

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a neural network with parameters ϕ that may be instantiated by any suitable sequence model. The learning objective then reads:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t | \mathbf{z})} \|\mu(\mathbf{y}_t^s, s | \mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

which we optimize by randomly drawing an observation at some time point t , together with its context window and covariates, from $\mathcal{D}$, and jointly minimizing $\mathcal{L}(\theta, \phi)$ over ϕ and θ with stochastic gradient descent. This corresponds to teacher forcing (Williams & Zipser, 1989), in which ground-truth past observations, rather than the model’s own predictions, are supplied during training, so that all time steps can be trained in parallel and error accumulation is avoided. Algorithm 1 summarizes the general training procedure for autoregressive flow matching.

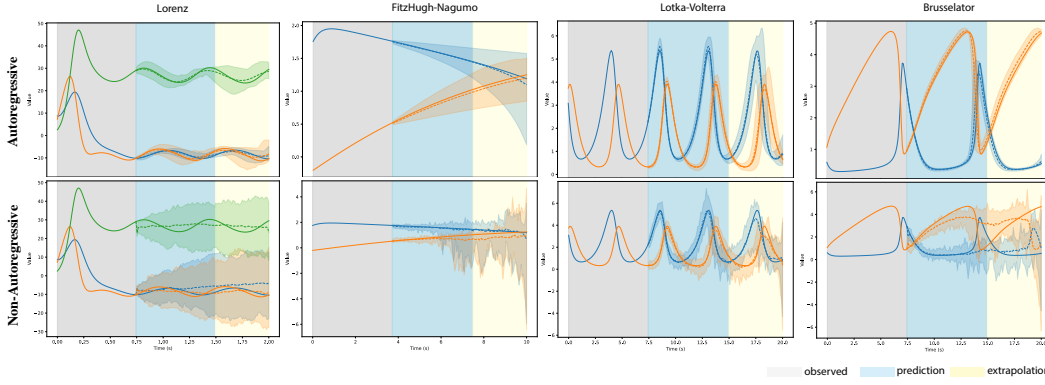


Figure 2: Representative forecasting results for autoregressive flow matching versus standard flow matching across four distinct dynamical systems, illustrating behavior that is consistent across system types. Solid lines denote ground truth, while dashed lines denote the mean prediction together with the 95% confidence interval. Results are shown for both the prediction and the extrapolation regimes.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
- 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
- 5: Compute target: $\mu(\mathbf{y}_t^s, s | \mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
- 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s | \mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
- 7: Update θ, ϕ
- 8: **end while**
- 9: **return** ν_θ, ζ_ϕ

3.3 INFERENCE

Once trained, the model can produce probabilistic predictions for any new observations $\mathbf{Y}_l$ and accompanying covariates $\mathbf{C}$ over an arbitrary number f of future timestamps. To this end we follow the sampling procedure in Algorithm 2, generating predictions autoregressively with a rolling-window approach. Because the same shared flow is reused at every step, the horizon f need not match any value seen during training. Repeating the procedure many times yields empirical quantiles that characterize the full predictive uncertainty.

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
- 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
- 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
- 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
- 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s), \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0, \quad s \in [0, 1]$
- 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
- 7: **end for**
- 8: **return** $\mathbf{Y}_f$

4 EXPERIMENTS

In this section, we present our empirical study and compare FLOWTIME against a broad set of baselines, using both real-world datasets and data simulated from a range of dynamical systems. Together these settings span diverse domains, dimensionalities, and dynamics, allowing us to probe how well the approach generalizes.

4.1 DYNAMICAL SYSTEMS

Data We evaluated FLOWTIME on forecasting five stochastic dynamical systems that together cover a wide spectrum of nonlinear behaviors, including chaotic, oscillatory, and limit-cycle dynamics: stochastic versions of the Lorenz system (Lorenz, 1963), the FitzHugh–Nagumo model (FitzHugh, 1961), the Lotka–Volterra system Lotka (1925), the Brusselator (Lefever & Prigogine, 1968), and the Van der Pol oscillator (van der Pol, 1926). For each system we generated trajectories by numerically solving the corresponding stochastic differential equations from randomly sampled initial conditions and Brownian motion realizations, so that the model is exposed to broad variability in dynamics. Details of the systems and the data-generation process are provided in Appendix A.1.

Implementation details For these experiments we instantiate ζ_ϕ as a two-layer bi-directional LSTM with 64 hidden units and ν_θ as an MLP with three hidden layers of size 64; these are deliberately lightweight, generic choices rather than architectures tailored to any particular system. We encode the flow step s into a 16-dimensional vector using Fourier positional embeddings with a fixed set of frequencies, and set the context length w equal to the prediction length. Models were trained with stochastic gradient descent using the Adam optimizer with a learning rate of 0.003 and a batch size of 128. Training to convergence took roughly 7 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines To isolate the contribution of the autoregressive factorization in FLOWTIME, we compare against a non-autoregressive flow-matching baseline that directly models the joint distribution of future observations given past observations and covariates, i.e. $p^1 = p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$. This comparison serves as a controlled ablation that holds every other design choice fixed, so that any difference can be attributed to the factorization itself. This non-autoregressive formulation closely resembles the method of (Kollovieh et al., 2024). Further details on the baseline are provided in Appendix B.

Evaluation To probe how well the model extrapolates beyond the conditions seen in training, we partition each trajectory into three segments: (1) an observation window that serves as historical input; (2) a prediction window that is available only during training as the target; and (3) an extrapolation window that is entirely withheld during training. We report the continuous rank probability score (CRPS) (Winkler et al., 1996) to assess the quality of uncertainty quantification and the normalized root mean square error (NRMSE) (Hyndman & Koehler, 2006) to assess point-prediction accuracy; both are standard, scale-comparable metrics that support consistent comparison across the diverse systems considered.

Results Our results, summarized in Table 1, show that the autoregressive factorization in FLOWTIME yields broad and consistent gains in forecasting performance across the full range of stochastic dynamical systems considered. On the prediction task, the AR approach improves NRMSE over the non-AR baseline for every one of the five systems, with especially large gains for the Lorenz (0.017 vs 0.242) and Brusselator (0.031 vs 0.658) systems, corresponding to error reductions of 93% and 95%. AR factorization also attains better CRPS in four of the five systems, with the Van der Pol oscillator the sole case in which the non-AR approach yields sharper uncertainty estimates. Crucially, these advantages carry over to the extrapolation regime, where AR factorization retains its NRMSE lead across all systems, indicating a robust ability to generalize well beyond the conditions seen in training. The effect is most striking for the Brusselator, where AR factorization cuts the extrapolation NRMSE by 98% (0.023 vs 1.046). Taken together, these results indicate that modeling temporal dependencies through autoregressive factorization allows FLOWTIME to capture the underlying dynamics of complex stochastic systems more faithfully, delivering more accurate pre-

Table 1: Performance comparison of autoregressive (FLOWTIME) versus non-autoregressive (Non-AR) factorization within our framework across five distinct stochastic dynamical systems. Performance is reported as CRPS and NRMSE for both the prediction and extrapolation regimes on the test set. Reported values are the mean and standard deviation over five random seeds, so the trends reflect behavior that is consistent across systems rather than a single run.

System	Factorization	Prediction		Extrapolation	
		CRPS ↓	NRMSE ↓	CRPS ↓	NRMSE ↓
Lorenz	Non-AR	0.147±0.012	0.242±0.018	0.120±0.009	0.301±0.024
	FLOWTIME	0.090±0.008	0.017±0.003	0.137±0.011	0.048±0.006
FitzHugh-Nagumo	Non-AR	0.165±0.014	0.131±0.011	0.214±0.016	0.306±0.022
	FLOWTIME	0.144±0.013	0.090±0.007	0.146±0.012	0.278±0.019
Lotka-Volterra	Non-AR	0.130±0.011	0.266±0.019	0.136±0.012	0.502±0.033
	FLOWTIME	0.126±0.010	0.189±0.015	0.123±0.009	0.339±0.026
Brusselator	Non-AR	0.217±0.016	0.658±0.042	0.238±0.019	1.046±0.057
	FLOWTIME	0.125±0.011	0.031±0.004	0.109±0.008	0.023±0.003
Van der Pol	Non-AR	0.143±0.012	0.161±0.014	0.211±0.018	0.510±0.031
	FLOWTIME	0.225±0.017	0.051±0.005	0.282±0.021	0.101±0.009

dictions and better-calibrated uncertainty across diverse dynamics, and particularly in the demanding extrapolation setting.

4.2 REAL-WORLD DATASETS

Data We evaluated FLOWTIME on forecasting a diverse collection of univariate and multivariate timeseries drawn from distinct application domains and spanning multiple sampling frequencies (hourly and daily), all obtained from GluonTS (Alexandrov et al., 2020). Specifically, we used the Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019) datasets, which together range from low- to high-dimensional series and cover energy, finance, transportation, and web-traffic domains. Further details are provided in Appendix A.2.

Implementation details Here we instantiate ζ_ϕ as a three-layer bi-directional LSTM with 64 hidden units and ν_θ as a residual network with five residual blocks built from 1-D convolutional layers with gated activation functions (Van Den Oord et al., 2016; Kong et al., 2020). This architecture follows the network of (Rasul et al., 2021) but uses fewer residual blocks, illustrating that the same overall recipe transfers across data regimes with only modest, standard adjustments. The flow step s is encoded into a 32-dimensional vector via Fourier positional embeddings with a fixed set of frequencies, and the same embedding scheme encodes the time-dependent covariates c . Models were trained with stochastic gradient descent using the Adam optimizer with a learning rate of 0.001 and a batch size of 128. Depending on the dataset, training to convergence took between 40 and 100 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines We compare FLOWTIME against a broad and representative set of established baselines spanning several methodological families. These include classical statistical methods such as Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008); deep learning methods such as DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016); diffusion-based methods such as TSDiff (Kolloviev et al., 2023), SSSD (Alcaraz & Strodthoff, 2022), and TimeGrad (Rasul et al., 2021); and the flow-matching method TSFlow (Kolloviev et al., 2024). This range of comparators lets us position our approach against the full breadth of contemporary forecasting paradigms.

Results Table 2 compares FLOWTIME against the full range of baselines on the five real-world timeseries datasets. Across these varied domains, FLOWTIME attains state-of-the-art performance

Table 2: Test-set CRPS comparison (lower is better) across a diverse set of models on five real-world datasets spanning multiple domains and dimensionalities. Mean and standard error for FlowTime are obtained by re-training and evaluating five times. Baseline results are taken from (Kollovich et al., 2024). Best scores in **bold**, second best underlined.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060±0.000	0.013±0.000	0.512±0.000	0.221±0.000	0.423±0.000
ARIMA	0.344±0.000	0.013±0.000	0.558±0.003	0.486±0.000	0.654±0.000
ETS	0.056±0.000	0.008±0.000	0.550±0.000	0.492±0.000	0.651±0.000
DeepAR	0.051±0.000	0.013±0.004	0.429±0.015	0.103±0.000	0.215±0.003
WaveNet	0.058±0.008	0.012±0.001	0.360±0.000	0.099±0.000	0.207 ±0.000
CSDI	0.051±0.000	0.013±0.000	0.360±0.000	0.152±0.000	0.318±0.012
SSSD	0.048±0.000	<u>0.010</u> ±0.000	0.354±0.024	0.107±0.002	<u>0.209</u> ±0.000
TSFlow	<u>0.045</u> ±0.001	0.009 ±0.001	<u>0.343</u> ±0.002	0.083 ±0.000	0.227±0.000
FLOWTIME	0.042 ±0.001	0.009 ±0.001	0.284 ±0.002	<u>0.089</u> ±0.000	0.243±0.002

on three of the five datasets: Electricity, Exchange, and Solar. On Electricity it surpasses every baseline with a CRPS of 0.042, a 6.7% improvement over the second-best model, TSFlow (0.045). On Exchange it matches the best result of TSFlow at a CRPS of 0.009 while clearly outperforming the classical statistical and other deep learning approaches. The gain is most pronounced on Solar, where FLOWTIME reaches a CRPS of 0.284, a 17.2% error reduction relative to the next-best TSFlow (0.343). On Traffic it ranks a close second at 0.089, marginally behind TSFlow (0.083) yet ahead of all remaining classical and deep learning baselines, and on Wikipedia it remains competitive at 0.243, trailing only WaveNet (0.207) and SSSD (0.209). Overall, these results indicate that the autoregressive flow-matching approach in FLOWTIME delivers consistently strong—state-of-the-art or highly competitive—performance across a broad and heterogeneous range of real-world forecasting tasks.

5 DISCUSSION

In this work, we introduced FLOWTIME, a general probabilistic forecasting framework that couples autoregressive modeling with flow matching to produce accurate and well-calibrated predictions for timeseries data ranging from univariate to high-dimensional multivariate. By factorizing forecasting into a sequence of conditional distributions, FLOWTIME balances the sequential structure of temporal data against the expressiveness of flow-based generative modeling, in a way that applies uniformly across domains, dimensionalities, and horizons. Our experiments across both simulated stochastic dynamical systems and real-world datasets show that FLOWTIME matches or exceeds state-of-the-art baselines on challenging forecasting benchmarks drawn from diverse application areas. The autoregressive decomposition yields clear gains in uncertainty calibration and distribution coverage over standard flow-matching approaches that model entire future windows at once. Just as importantly, FLOWTIME generalizes robustly to out-of-distribution conditions—a capability essential for real-world deployment, where test settings routinely depart from those seen during training.

Several limitations point to broader opportunities. First, the sequential nature of autoregressive sampling adds inference-time overhead relative to fully parallelizable non-autoregressive models, which may constrain use in latency-sensitive, real-time systems. Second, our current implementation applies flow matching directly in the data space, which can be suboptimal for very high-dimensional or partially observed data. These observations suggest a rich agenda for future work: latent-space formulations, extensions to irregularly sampled and asynchronous data, and applications reaching well beyond the domains studied here into scientific modeling and decision-making under uncertainty more broadly. Overall, FLOWTIME advances probabilistic timeseries forecasting and, more generally, illustrates the promise of flow-based generative models for jointly improving predictive accuracy and uncertainty quantification across a wide range of settings.

REFERENCES

Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.

432 Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and fore-
 433 casting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
 434
 435 Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan
 436 Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper
 437 Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of*
 438 *Machine Learning Research*, 21(116):1–6, 2020.
 439
 440 Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary
 441 differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
 442
 443 Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey.
 444 In *ArXiv:2104.00164*, 2021.
 445
 446 Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-
 447 encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
 448
 449 Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>,
 450 7(1):62, 2017.
 451
 452 Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam
 453 Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for
 454 high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*,
 455 2024.
 456
 457 Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane.
 458 *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.
 459
 460 Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas,
 461 Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function
 462 RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–
 463 1910. PMLR, 2019.
 464
 465 Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFIORD:
 466 Free-Form continuous dynamics for scalable reversible generative models. In *International Con-*
 467 *ference on Learning Representations*, 2018.
 468
 469 Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in*
 470 *Neural Information Processing Systems*, 33:6840–6851, 2020.
 471
 472 Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen.
 473 Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL
 474 <http://arxiv.org/abs/2411.07506>.
 475
 476 Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential*
 477 *Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.
 478
 479 Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International*
 480 *Journal of Forecasting*, 22(4):679–688, 2006.
 481
 482 Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep varia-
 483 tional Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint*
 484 *arXiv:1605.06432*, 2016.
 485
 486 Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang,
 487 and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for proba-
 488 bilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–
 489 28364, 2023.
 490
 491 Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow
 492 matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth*
 493 *International Conference on Learning Representations*, 2024.

486 Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile
487 diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.
488

489 Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term
490 temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference*
491 *on Research & Development in Information Retrieval*, pp. 95–104, 2018.

492 R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal*
493 *of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.
494

495 Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical*
496 *Transactions of the Royal Society A*, 379(2194):20200209, 2021.

497 Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow match-
498 ing for generative modeling. In *The Eleventh International Conference on Learning Representa-*
499 *tions*, 2022.

500

501 Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data
502 with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.
503

504 Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.

505 Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.

506

507 Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv*
508 *preprint arXiv:2401.03006*, 2024.

509

510 Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv
511 Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media founda-
512 tion models. *arXiv e-prints*, pp. arXiv–2410, 2024.

513

514 Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Mul-
515 tivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint*
516 *arXiv:2002.06103*, 2020.

517

518 Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising dif-
519 fusion models for multivariate probabilistic time series forecasting. In *International Conference*
520 *on Machine Learning*, pp. 8857–8868. PMLR, 2021.

521

522 David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus.
523 High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances*
524 *in Neural Information Processing Systems*, 32, 2019.

525

526 David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic
527 forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):
528 1181–1191, 2020.

529

530 Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised
531 learning using nonequilibrium thermodynamics. In *International Conference on Machine Learn-*
532 *ing*, pp. 2256–2265. PMLR, 2015.

533

534 Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben
535 Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint*
536 *arXiv:2011.13456*, 2020.

537

538 Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow match-
539 ing for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Gener-*
ative Modeling, 2024.

Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based
diffusion models for probabilistic time series imputation. *Advances in Neural Information Pro-*
cessing Systems, 34:24804–24816, 2021.

Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Hugué, Yanlei Zhang, Jarrod Rector-Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.

B. van der Pol. On 'relaxation-oscillations'. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.

Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, 1(2):270–280, 1989.

Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.

Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time series foundation models: Generalizing time series representation with large language model. In *ArXiv:2104.00164*, 2024.

A DATASETS

A.1 DYNAMICAL SYSTEMS

We generated training and testing data for five classical stochastic dynamical systems—Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator, and Van der Pol—by numerically integrating their stochastic differential equations (SDEs). These systems were chosen to span a wide variety of qualitative dynamics, and each shares the general form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \Theta) dt + \sigma d\mathbf{W}(t),$$

where f is the deterministic drift, Θ the system parameters, σ a constant diffusion term, and $\mathbf{W}(t)$ a standard Brownian motion. For each system we drew 2400 random initial conditions from a uniform distribution over the specified range (see Table 3), thereby covering a broad portion of each system’s state space, and generated one trajectory per initial condition by solving the SDE with the Euler-Heun method at a fixed step size. Each trajectory was sampled at 200 equally spaced time points, of which the first 75 served as observed data, the next 75 as the prediction target, and the final 50 as the held-out extrapolation window. Of the 2400 trajectories, 2000 were used for training and 400 for testing.

Table 3: Summary of the stochastic dynamical systems used in our study. $\mathbf{f}(\mathbf{x})$ denotes the drift function, Θ the system parameters, σ the constant diffusion term, and $p(\mathbf{x}(0))$ the initial-condition distribution. Each system was integrated over the stated interval $[t_0, t_1]$ with 200 steps using the Euler-Heun method, spanning a diverse set of nonlinear dynamics.

System	$f(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh-Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka-Volterra		$I = 0.5$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\alpha = 1.3$			
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\beta = 0.9$			
Brusselator		$\gamma = 0.8$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$\delta = 1.8$			
	$f_2 = B x_1 - x_1^2 x_2$	$A = 1.0$			
Van der Pol	$f_1 = x_2$	$B = 3.0$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$	$\mu = 0.1$			

A.2 REAL-WORLD DATASETS

To evaluate our model across a range of domains and scales, we used five datasets from GluonTS (Alexandrov et al., 2020), adopting the preprocessed versions of (Salinas et al., 2019). A

Table 4: Properties of the real-world datasets used in the experiments, illustrating the range of scales and domains they cover: dimension n , domain Ω , sampling frequency, total training timesteps, and prediction length.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

summary of their properties, highlighting their varied dimensionalities and frequencies, is given in Table 4.

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting We adopt the same general forecasting setup as Section 3: given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$, the goal remains to model the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. In contrast to the autoregressive factorization, however, here we model the joint distribution of the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$ directly, without any temporal factorization:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation preserves temporal correlations across all future time steps, at the cost of having to learn a single high-dimensional distribution.

Training We construct a probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ that transports samples from a prior $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ to the target $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$, where Σ is a block-diagonal covariance matrix. The prior admits a natural interpretation as independent per-dimension Brownian motion processes, with Brownian motion defined as a stochastic process $W: [\tau+1, \tau+f] \rightarrow \mathbb{R}^n$.

We define the conditional probability path using a linear interpolation bridge with Brownian noise:

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$ with $\mathbf{Y}_f^0 \sim p^0$ and $\mathbf{Y}_f^1 \sim p^1$. The corresponding conditional velocity field becomes:

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

As $\sigma^2 \rightarrow 0$, this reduces to the straight-path velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. We then learn a neural velocity field ν_θ acting on the entire future trajectory. The context encoding mirrors Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ now ingests both past and future covariates. The training objective becomes:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \|\mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|^2 \quad (13)$$

where ν_θ can be implemented by any sequential neural network (e.g., a Transformer or RNN) that processes the full trajectory. Sampling then requires solving the trajectory-level ODE:

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

with a standard numerical solver. This yields joint samples from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ without any autoregressive decomposition.

Implementation Details We used this baseline for the dynamical-systems forecasting experiments across all five systems. Here we instantiate ν_θ as a 4-layer bi-directional LSTM with 128 hidden units and retain the training parameters of Section 4.1 unchanged, so that the comparison remains controlled. Training to convergence took roughly 2 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Key Differences from Autoregressive Variant The non-autoregressive approach differs fundamentally from the autoregressive variant of FLOWTIME along several general axes. First, whereas the autoregressive method factorizes the joint distribution through a Markovian decomposition across time steps, the non-autoregressive baseline models the full future trajectory $\mathbf{Y}_f$ as a single high-dimensional random variable, preserving cross-temporal dependencies at the cost of learning a more complex distribution over $\mathbb{R}^{n \times f}$. Second, their computational profiles diverge markedly: the non-autoregressive version pushes entire trajectories through sequential velocity-field networks,

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
- 3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$
- 5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
- 6: Compute target $\mu(\mathbf{Y}_f^s, s|\mathbf{z})$
- 7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
- 8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ

- 1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
- 2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
- 3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
- 4: Return $\psi(\mathbf{Y}_f, 1)$

enabling parallel generation of all future timepoints but incurring higher memory requirements. Finally, their uncertainty-propagation mechanisms differ in kind: the autoregressive approach explicitly models how prediction errors accumulate along the chain of conditional distributions, whereas the non-autoregressive method captures joint uncertainty over all timesteps through trajectory-level sampling without an explicit model of error-accumulation dynamics. These distinctions give rise to complementary trade-offs that span the design space—the autoregressive variant offers interpretable uncertainty quantification and efficient window-based computation under a limited Markovian assumption, while the non-autoregressive baseline preserves full temporal correlations at higher computational cost with less explicit uncertainty dynamics.

C EVALUATION METRICS

This section describes the metrics used to assess forecast quality. Given an n -dimensional time series $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ of arbitrary dimension together with its predictive distribution or samples from our probabilistic model, we employ two widely adopted and complementary metrics: the Continuous Ranked Probability Score (CRPS) and the Normalized Root Mean Squared Error (NRMSE).

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

The CRPS is a proper scoring rule widely used to assess probabilistic forecasts across many domains. For a univariate random variable X with cumulative distribution function (CDF) F_X and observed value x , it is defined as:

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ is the indicator function. Intuitively, the CRPS measures how well the entire predictive distribution aligns with the observation x , and lower values indicate better-calibrated, more accurate predictive distributions.

For time series forecasting, we compute the CRPS at each step of the horizon and average over the horizon:

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ is the predicted CDF (or its empirical counterpart from samples) at horizon step k and $y_{\tau+k}$ is the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

The Root Mean Squared Error (RMSE) is a standard, broadly applicable metric for the accuracy of point forecasts. For predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ and corresponding ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, it is given by:

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

To render the RMSE scale-invariant and thus comparable across datasets and time series of widely differing magnitudes, we use the Normalized RMSE (NRMSE). A common normalization divides by the standard deviation of the ground-truth series, σ_y :

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observations. Depending on the application, one may equally normalize by the data range ($\max(y) - \min(y)$). In all cases, a lower NRMSE reflects more accurate predictions relative to the intrinsic variability of the time series.